

BÁO CÁO ĐỒ ÁN / TIỂU LUẬN

DỰ ÁN E-COMMERCE MICROSERVICE

Phân tích use-case, thiết kế hệ thống, hiện thực mã nguồn và kết quả triển khai

Họ và tên: Nguyễn Tuấn Anh

Lớp: E22CNPM-01

Mã sinh viên: B22DCDT019

Đề tài: [E-commerce Microservice]

Ngày 14 tháng 4 năm 2026

BÁO CÁO DỰ ÁN E-COMMERCE MICROSERVICE

Phân tích use-case, thiết kế hệ thống, hiện thực mã nguồn và kết quả triển khai

Ngày 14 tháng 4 năm 2026

Tóm tắt nội dung

Tài liệu này trình bày báo cáo kỹ thuật cho dự án `e-commerce-microservice`. Nội dung đi từ bối cảnh nghiệp vụ, use-case cốt lõi, kiến trúc microservice, cấu trúc hiện thực trong mã nguồn đến kết quả triển khai bằng Docker Compose. Hệ thống mô phỏng một nền tảng thương mại điện tử gồm quản lý người dùng, danh mục sản phẩm, đơn hàng, thanh toán, giao diện điều hành và một dịch vụ AI hỗ trợ truy vấn dữ liệu nghiệp vụ.

1. Giới thiệu dự án và bài toán

Trong bối cảnh chuyển đổi số, các hệ thống thương mại điện tử thường phải xử lý nhiều miền nghiệp vụ độc lập như khách hàng, sản phẩm, đơn hàng, thanh toán và phân tích hỗ trợ ra quyết định. Nếu tất cả được đặt trong một ứng dụng nguyên khối, phạm vi thay đổi sẽ ngày càng lớn, việc bảo trì khó khăn và năng lực mở rộng bị hạn chế.

Dự án `e-commerce-microservice` được xây dựng để mô phỏng một hệ thống e-commerce hiện đại theo hướng microservice. Mục tiêu của dự án không chỉ là cung cấp API CRUD đơn thuần mà còn thể hiện rõ cách các dịch vụ giao tiếp với nhau thông qua gateway, cách frontend tiêu thụ dữ liệu hợp nhất, và cách bổ sung một thành phần AI để hỗ trợ tra cứu thông tin nghiệp vụ.

1.1. Use-case chính của hệ thống

Hệ thống xoay quanh các kịch bản nghiệp vụ sau:

- Quản trị viên hoặc nhân viên xem nhanh tình trạng toàn hệ thống qua dashboard.
- Người dùng được quản lý theo vai trò `admin`, `customer`, `staff`.

- Sản phẩm được lưu trong catalog với thông tin giá, giảm giá, tồn kho, đánh giá và thuộc tính mở rộng.
- Đơn hàng được tạo từ dữ liệu người dùng và danh sách sản phẩm, trong đó `order_service` phải kiểm tra tính hợp lệ của người dùng và sản phẩm trước khi ghi dữ liệu.
- Thanh toán được tạo dựa trên đơn hàng đã tồn tại, do `payment_service` xác nhận từ `order_service`.
- Dashboard và chatbot AI cho phép quan sát nhanh dữ liệu nghiệp vụ, truy vấn đơn hàng, sản phẩm và khách hàng.

2. Mục tiêu thiết kế

Các mục tiêu thiết kế chính của dự án gồm:

- Tách biệt rõ từng miền nghiệp vụ thành các service độc lập.
- Sử dụng API gateway làm điểm vào duy nhất cho frontend.
- Cho phép triển khai toàn bộ hệ thống bằng một lệnh `docker compose up -build`.
- Duy trì khả năng quan sát qua các endpoint tổng hợp như `/summary/`.
- Kết hợp frontend Next.js và backend Django/FastAPI theo một luồng end-to-end dễ demo.

3. Kiến trúc tổng thể

3.1. Thành phần hệ thống

Theo cấu hình trong `docker-compose.yaml`, hệ thống gồm bảy thành phần:

Thành phần	Vai trò
<code>client</code>	Frontend Next.js, hiển thị dashboard, tìm kiếm sản phẩm, tạo đơn hàng và tạo thanh toán.
<code>gateway</code>	API gateway bằng Django, nhận request từ client và proxy tới các service nghiệp vụ.
<code>user_service</code>	Quản lý hồ sơ người dùng, phân vai trò, đăng nhập demo và thống kê người dùng.
<code>product_service</code>	Quản lý catalog sản phẩm, hỗ trợ lọc theo danh mục và tổng hợp dữ liệu catalog.
<code>order_service</code>	Tạo đơn hàng, hydrate thông tin người dùng và sản phẩm, tính subtotal, shipping fee và tổng tiền.

<code>payment_service</code>	Tạo bản ghi thanh toán cho đơn hàng, sinh mã giao dịch và thống kê giá trị đã xử lý.
<code>ai_service</code>	Dịch vụ FastAPI cho chatbot intent classification và truy vấn dữ liệu dạng retrieval.
<code>postgres</code>	PostgreSQL dùng chung một instance, mỗi service dùng schema riêng.

3.2. Luồng giao tiếp

Luồng request chính của hệ thống được tổ chức như sau:

1. Trình duyệt truy cập frontend tại cổng 3000.
2. Frontend gửi yêu cầu qua route handler nội bộ `/api/backend/*`.
3. Route handler tiếp tục chuyển tiếp sang `gateway`.
4. Gateway dựa trên tên service trong URL để chuyển tiếp tới `user_service`, `product_service`, `order_service`, `payment_service` hoặc `ai_service`.
5. Các service nghiệp vụ giao tiếp đồng bộ với nhau bằng HTTP khi cần xác thực dữ liệu liên service.
6. Dữ liệu cuối cùng được lưu trong PostgreSQL.

4. Thiết kế nghiệp vụ và dữ liệu

4.1. Thiết kế người dùng

`user_service` quản lý thực thể `UserProfile`. Mô hình dữ liệu gồm các thuộc tính nghiệp vụ cơ bản như họ tên, email, số điện thoại, vai trò, trạng thái, điểm loyalty, địa chỉ và metadata mở rộng. Việc dùng `role` theo `TextChoices` giúp chuẩn hóa các nhóm người dùng trong toàn hệ thống.

4.2. Thiết kế sản phẩm

`product_service` lưu thực thể `Product` với cấu trúc giàu thuộc tính: SKU, danh mục, thương hiệu, giá, giá giảm, tồn kho, an toàn tồn kho, kích thước, xuất xứ, bảo hành, tags, attributes và thumbnail. Thiết kế này thể hiện tư duy catalog linh hoạt, phù hợp với bài toán thương mại điện tử.

4.3. Thiết kế đơn hàng

`order_service` sử dụng hai bảng:

- **Order:** lưu thông tin khách hàng, địa chỉ giao, trạng thái, subtotal, phí vận chuyển, tổng tiền và metadata.
- **OrderItem:** lưu snapshot của sản phẩm tại thời điểm mua, bao gồm tên, SKU, số lượng, đơn giá và thành tiền.

Việc lưu `product_snapshot` là quyết định thiết kế quan trọng vì giúp giữ nguyên dấu vết nghiệp vụ ngay cả khi catalog thay đổi sau đó.

4.4. Thiết kế thanh toán

`payment_service` lưu thực thể `Payment` với các trường `order_id`, `amount`, `method`, `status`, `transaction_code`, `gateway_response` và `paid_at`. Thiết kế này mô phỏng gần với hệ thống thực tế ở mức demo, đặc biệt là trường `gateway_response` cho phép giữ metadata từ cổng thanh toán.

5. Thiết kế mã nguồn

5.1. Gateway làm điểm vào duy nhất

Gateway được cài đặt bằng Django view đơn giản nhưng hiệu quả. Hàm `proxy_router` nhận tham số `service`, kiểm tra xem service có hợp lệ không, sau đó dựng URL upstream và chuyển tiếp nguyên phương thức HTTP, header và body tới service đích.

Listing 1: Ý tưởng proxy tại gateway

```
def proxy_router(request, service, subpath=""):
    if service not in settings.SERVICE_URLS:
        return JsonResponse({"error": "Unsupported service"}, status=404)

    upstream = urljoin(settings.SERVICE_URLS[service], subpath.lstrip("/"))
    response = requests.request(
        method=request.method,
        url=upstream,
        headers=headers,
        data=request.body or None,
        timeout=20,
    )
    return HttpResponse(content=response.content, status=response.status_code)
```

Thiết kế này đem lại ba lợi ích:

- Frontend chỉ cần biết một backend base URL.
- Có thể thay đổi vị trí hoặc số lượng service mà không phải sửa logic client nhiều nơi.
- Thuận tiện để chèn thêm xác thực, logging hay rate limiting trong tương lai.

5.2. Tạo đơn hàng bằng kiểm tra liên service

Phần quan trọng nhất của `order_service` là luồng tạo đơn hàng. Khi nhận yêu cầu mới, service:

1. Gọi `user_service` để kiểm tra `user_id`.
2. Duyệt từng item và gọi `product_service` để lấy dữ liệu sản phẩm.
3. Tính `line_total`, `subtotal`, `shipping_fee` và `total_amount`.
4. Tạo `Order` trước, sau đó `bulk_create` các `OrderItem`.

Listing 2: Rút gọn logic tạo đơn hàng

```
for item in items_payload:
    product = _fetch_product(item["product_id"])
    quantity = int(item["quantity"])
    unit_price = Decimal(str(product["current_price"]))
    line_total = unit_price * quantity
    subtotal += line_total

order = Order.objects.create(
    user_id=user["id"],
    subtotal_amount=subtotal,
    shipping_fee=shipping_fee,
    total_amount=subtotal + shipping_fee,
)
```

Điểm đáng chú ý là service không dùng foreign key xuyên service. Tính đúng đắn nghiệp vụ được đảm bảo bằng HTTP call ở thời điểm xử lý.

5.3. Thanh toán dựa trên đơn hàng hiện có

`payment_service` áp dụng cách tiếp cận tương tự. Trước khi tạo thanh toán, hệ thống gọi `order_service` để lấy dữ liệu đơn hàng. Sau đó `payment` được sinh với `transaction_code` ngẫu nhiên và `metadata` trả về từ gateway demo.

Listing 3: Tạo thanh toán từ đơn hàng

```
order = _fetch_order(payload["order_id"])
payment = Payment.objects.create(
    order_id=order["id"],
    payer_name=order["customer_name"],
    amount=order["total_amount"],
    method=payload.get("method", Payment.PaymentMethod.MOMO),
    transaction_code=uuid.uuid4().hex[:16].upper(),
)
```

5.4. Frontend Next.js và lớp proxy nội bộ

Frontend được viết bằng Next.js. Một route handler tại `src/app/api/backend/[...path]/route.ts` đóng vai trò proxy lớp thứ hai. Thành phần này rất hữu ích khi frontend cần gọi backend từ cả server side lẫn client side mà vẫn giữ cùng một domain logic.

Listing 4: Proxy nội bộ trong Next.js

```
const backendBaseUrl =
  process.env.ECOMMERCE_API_BASE_URL?.replace(/\/$/, "")
  ?? "http://127.0.0.1:8000";

const target = `${backendBaseUrl}/api/${backendPath}${search}`;
const response = await fetch(target, {
  method: request.method,
  body: request.method === "GET" ? undefined : await request.text(),
  cache: "no-store",
});
```

Dashboard phía client đồng thời tải nhiều nguồn dữ liệu bằng `Promise.all`, ví dụ user summary, product summary, order summary và payment summary. Đây là thiết kế hợp lý để giảm thời gian chờ giao diện.

5.5. Dịch vụ AI hỗ trợ truy vấn nghiệp vụ

Ngoài các service CRUD, dự án còn có `ai_service` viết bằng FastAPI. Thành phần này:

- Nạp mô hình intent classification và vectorizer từ file pickle.
- Kết hợp rule-based intent detection với fallback model.
- Thực hiện retrieval cho dữ liệu product, order và customer.
- Trả về phản hồi có `sources` để giải thích kết quả.

Đây là điểm mở rộng đáng giá vì thể hiện xu hướng tích hợp AI như một service độc lập thay vì nhúng trực tiếp vào backend nghiệp vụ.

6. Triển khai và vận hành

6.1. Môi trường triển khai

Dự án được đóng gói bằng Docker Compose. Các cổng mặc định:

- Client: `http://localhost:3000`
- Gateway: `http://localhost:8000`

- PostgreSQL: localhost:5433

Lệnh chạy chuẩn:

```
cd e-commerce-microservice
docker compose up --build
```

6.2. Kết quả đạt được

Từ cấu trúc hiện có trong mã nguồn, dự án đã đạt được các kết quả chính:

- Xây dựng thành công hệ thống e-commerce đa dịch vụ với luồng nghiệp vụ xuyên service.
- Chuẩn hóa điểm vào backend thông qua gateway.
- Hỗ trợ dashboard quan sát dữ liệu người dùng, sản phẩm, đơn hàng và thanh toán.
- Cung cấp các endpoint summary phục vụ thống kê nhanh như tổng số người dùng, doanh thu, top customer và best-selling product.
- Tích hợp dịch vụ AI hỗ trợ tra cứu và chatbot nghiệp vụ.
- Triển khai toàn bộ hệ thống qua một file Compose duy nhất, giảm độ phức tạp vận hành.

6.3. Đánh giá kỹ thuật

Ưu điểm của dự án:

- Phân tách service rõ ràng theo bounded context.
- Dữ liệu đơn hàng và thanh toán được snapshot hóa hợp lý.
- Luồng frontend đến backend thống nhất, dễ demo và dễ mở rộng.
- Có thêm service AI làm nổi bật hướng phát triển hiện đại.

Giới hạn hiện tại:

- Các lời gọi liên service đang là đồng bộ, chưa có retry hoặc circuit breaker.
- Chưa có cơ chế xác thực tập trung và phân quyền API ở gateway.
- Cơ sở dữ liệu vẫn dùng chung một PostgreSQL instance, phù hợp cho demo hơn là production quy mô lớn.
- Test tự động trong repository còn mỏng.

7. Kết luận

`e-commerce-microservice` là một dự án minh họa tốt cho cách xây dựng nền tảng thương mại điện tử theo kiến trúc microservice với mức độ hoàn chỉnh tương đối cao cho mục đích học thuật và demo kỹ thuật. Dự án không chỉ giải quyết use-case quản lý người dùng, catalog, đơn hàng và thanh toán mà còn cho thấy cách phối hợp frontend, gateway, backend service và AI service trong cùng một hệ thống.

Nếu tiếp tục phát triển, dự án có thể mở rộng theo các hướng như xác thực JWT tập trung, message broker cho event-driven workflow, observability đầy đủ, tách database riêng cho từng service và tăng độ sâu cho AI assistant.